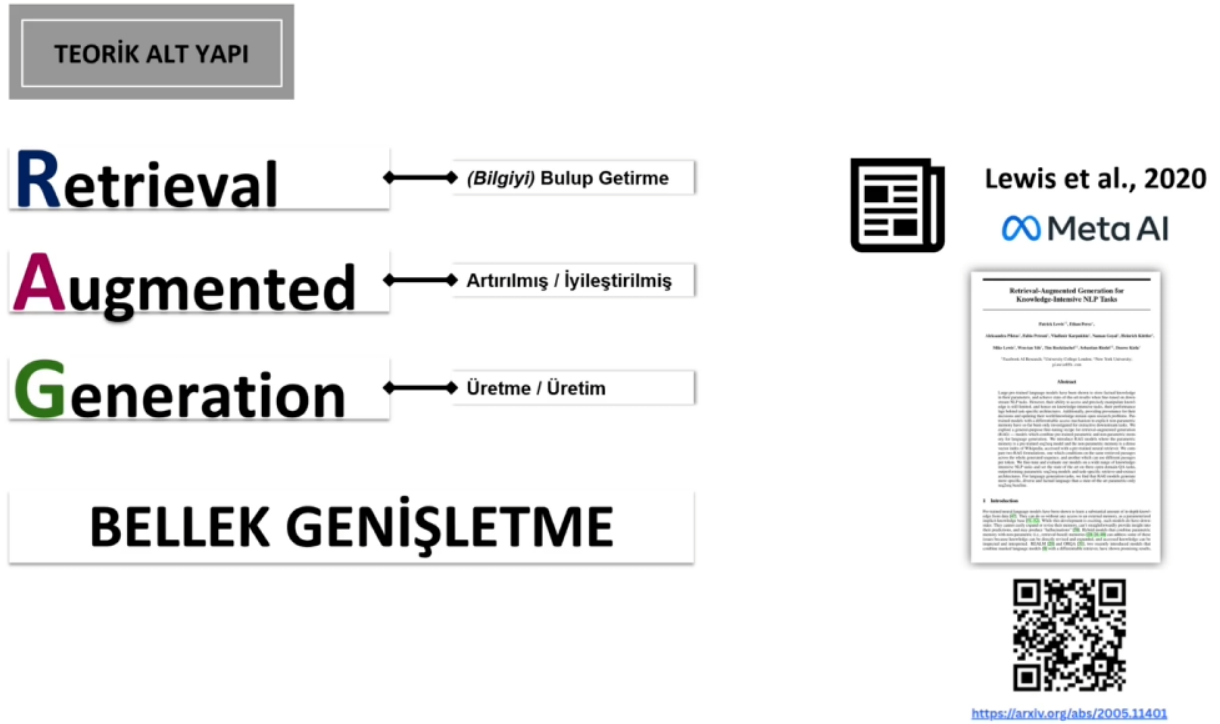


Bellek genişletme (RAG) süreçlerinde şimdiye kadar kodlama yaparak pratik adımları görmüştük. Şimdi, bu mimarinin bileşenlerini ve aralarındaki ilişkiyi tek bir büyük resimde inceleme vaktidir. Bu mimari ilk kez 2020 yılında **Meta AI** araştırmacıları tarafından önerilmiş ve günümüze kadar sayısız varyantı geliştirilmiştir.

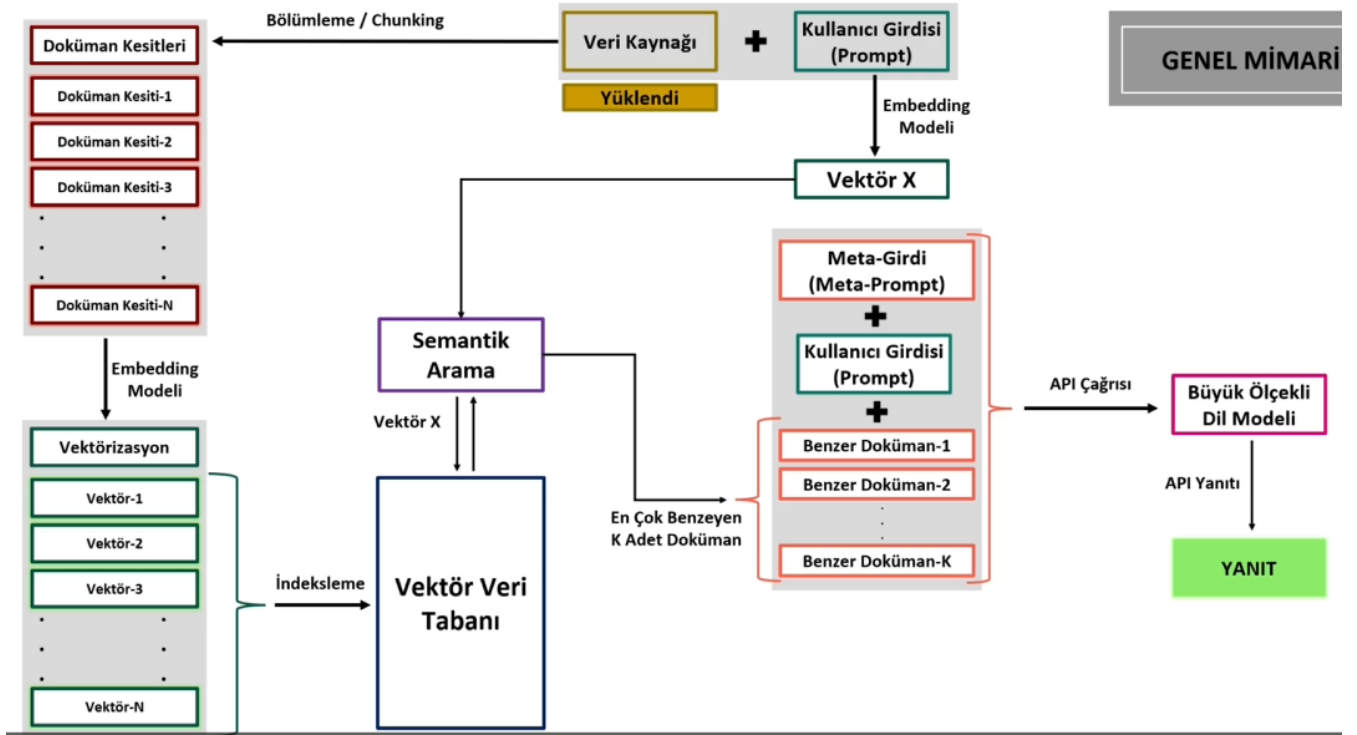
1. Teorik Altyapı ve Tarihçe

RAG mimarisinin temelleri, Lewis ve ekibinin 2020 yılında yayınladığı makaleye dayanmaktadır. Bu yaklaşımın özü, modelin kendi eğitim verisinde yer almayan veya öne çıkmayan bilgileri, dış bir veri kaynağı yardımıyla modelin odak noktasına getirmektir.



2. Genel Mimari ve İş Akışı

RAG mimarisini bir "başlangıç" ve "bitiş" koşulu olan sekiz temel aşamaya bölebiliriz.



I. Hazırlık Safhası (Veriden Vektöre)

- Veri Kaynağı & Yükleme:** Süreç, bir kullanıcı girdisi (prompt) ve bir veri kaynağı (PDF, web adresi, video vb.) ile başlar. İlk adımda, WebBaseLoader gibi sınıflarla veri sisteme yüklenir.
- Bölümleme (Chunking):** Veri kaynağı, daha yönetilebilir olması için "n" tane küçük kesite (chunk) ayrılır.
- Vektörizasyon (Embedding):** Dil modelleri metinlerle değil sayılarla çalışır. Bu nedenle her bir kesit, bir embedding modeli vasıtasıyla matematiksel birer vektöre dönüştürülür.
- İndeksleme & Vektör Veri Tabanı:** Oluşturulan "n" tane vektör, hızlı sorgulanabilmeleri için bir vektör veri tabanında indekslenerek saklanır.

II. Etkileşim Safhası (Sorgudan Yanıta)

- Kullanıcı Girdisinin Vektörleştirilmesi:** Kullanıcının promptu da aynı embedding modeliyle bir vektöre (Vektör X) dönüştürülür.
- Semantik Arama (Benzerlik Karşılaştırması):** Vektör X, veri tabanındaki tüm vektörlerle kıyaslanır. Matematiksel olarak Vektör X'e en yakın (en uyumlu) olan "k" adet doküman kesiti tespit edilir.
- Meta-Girdi (Meta-Prompt) İnşası:** Orijinal prompt ve semantik arama ile bulunan ilgili doküman kesitleri birleştirilir. Modelden, yanıtını sadece bu ek bilgiler ışığında vermesi istenir.

8. **LLM API Çağrısı ve Yanıt:** Hazırlanan meta-prompt büyük ölçekli dil modeline (LLM) gönderilir ve bellek genişletme sürecinden süzölmüş, isabetli yanıt alınır.

3. Kavramsal Yapı ve Kod Eşleşmesi

Daha önce yazdığımız LangChain tabanlı kod blokları, aslında bu teorik şemanın birer yansımasıdır.

Mimari Adım	LangChain Kod Karşılığı	İşlevi
Yükleme	WebBaseLoader(url).load()	Veriyi sisteme dahil eder.
Bölümlleme	RecursiveCharacterTextSplitter	Metni parçalara ayırır.
Vektörizasyon	OpenAIEmbeddings()	Metni sayı dizilerine çevirir.
Semantik Arama	retriever.get_relevant_documents()	En benzer "k" dökümanı bulur.
Meta-Prompt	final_prompt = f"{context} {question}"	Bilgi ve soruyu birleştirir.

Sonuç

RAG mimarisi, dil modellerinin halüsinasyon ve güncellik gibi kısıtlarını aşmak için en optimal çözümdür. Bu yapıda harcanan emek ile elde edilen isabetlilik oranı, profesyonel yapay zeka uygulamalarının temelini oluşturur.